

Deep Research: Avatrada 57 Topic 016

Engineering Report: 3-Stage Panic Protocol

Report ID: AVTRD-57-PANIC-001 **Date:** 2023-10-27 **Author:** Autonomous Technical Researcher **Subject:** Deep-Dive Analysis of a 3-Stage Emergency Position Liquidation Protocol

This report provides a comprehensive engineering analysis of the 3-Stage Panic Protocol specified in the source document. The analysis covers the protocol's technical architecture, a practical implementation strategy in Python using `asyncio`, and a critical evaluation of potential failure modes and optimizations.

1. Technical Deconstruction

The Panic Protocol is a pre-defined, automated sequence designed for rapid and guaranteed liquidation of a trading position under emergency conditions (e.g., risk limit breach, system failure, critical market event). Its architecture follows a principle of "graceful degradation," starting with an optimal, low-impact order and escalating in aggression to ensure final execution.

Stage 1: Snap to Mid (Passive Liquidation Attempt)

- **Mechanism:** A `LIMIT` order is placed at the exact mid-price of the current bid-ask spread.
- **Formula:** $\text{Limit_Price_1} = (\text{Best_Bid} + \text{Best_Ask}) / 2$
- **Rationale:** This is the most favorable execution scenario. For a sell order, it aims to capture the spread rather than paying it. It's a passive order, hoping a counterparty will aggressively cross the spread and fill it. This minimizes market impact and achieves the best possible price.

- **Duration:** 500ms. This is a very short window, indicating the system will not wait long for this ideal outcome. If the order is not filled almost instantly, the protocol assumes it won't be and escalates.

Stage 2: Limit Sweep (Aggressive, Price-Protected Liquidation)

- **Mechanism:** The Stage 1 order is cancelled, and a new, more aggressive **LIMIT** order is placed. The price is set significantly through the spread, deep into the order book.
- **Formula (for a SELL order):** $\text{Limit_Price_2} = \text{Best_Bid} * 0.95$
- **Rationale:** The passive attempt failed. The priority now shifts from optimal price to high-probability execution, while still maintaining a price floor. By placing a sell order 5% below the best bid, the intent is to "sweep" through the top layers of the order book, guaranteeing a fill against existing buy orders. The 5% buffer acts as a slippage control, preventing catastrophic execution in a "flash crash" scenario.
- **Duration:** 2 seconds. A longer duration is allocated because this order is expected to be filled. If it remains unfilled after 2 seconds, it implies extreme market illiquidity or a rapidly collapsing market where the price has moved away faster than the order could be executed.

Stage 3: Market Order (Guaranteed Liquidation)

- **Mechanism:** Any pending limit order from Stage 2 is cancelled, and a **MARKET** order is submitted for the remaining quantity.
 - **Formula:** Not applicable (Order Type, not a price).
 - **Rationale:** This is the final, non-negotiable stage. Both prior attempts have failed, and the system's primary directive is now to exit the position *at any cost*. A market order guarantees execution against any available liquidity, offering no price protection. It is the most aggressive action and has the highest potential for slippage and market impact, but it provides the highest certainty of exit.
-

2. Implementation Strategy

The implementation requires an asynchronous architecture to handle time-based waits without blocking other system processes. Python's `asyncio` library is perfectly suited for this I/O-bound task.

Core Components

1. **Asynchronous Exchange Client:** A wrapper for the exchange's API that supports `async/await` syntax for network requests (e.g., placing orders, checking status, fetching tickers).
2. **State Management:** A mechanism to ensure the panic protocol for a specific asset does not run concurrently or conflict with other trading logic.
3. **The `panic_exit` Function:** The core coroutine that orchestrates the 3-stage sequence.

Python Implementation

```
import asyncio
import logging
from decimal import Decimal

# --- Assume this is a simplified, pre-existing exchange client ---
class AsyncExchangeClient:
    async def get_ticker(self, symbol: str) -> dict:
        # Returns {'bid': Decimal('...'), 'ask': Decimal('...')}
        pass

    async def place_limit_order(self, symbol: str, side: str, quantity:
Decimal, price: Decimal) -> dict:
        # Returns an order confirmation dict, e.g., {'id': '12345'}
        pass

    async def place_market_order(self, symbol: str, side: str, quantity:
Decimal) -> dict:
        # Returns an order confirmation dict
        pass

    async def get_order_status(self, order_id: str, symbol: str) -> dict:
        # Returns {'status': 'filled'/'open'/'canceled', 'filled_quantity':
```

```

Decimal('...')}}
    pass

    async def cancel_order(self, order_id: str, symbol: str) -> bool:
        # Returns True on success
        pass

# --- Panic Protocol Implementation ---

class TradingSystem:
    def __init__(self, exchange_client: AsyncExchangeClient):
        self.client = exchange_client
        # A dictionary of locks to ensure atomicity on a per-symbol basis
        self._position_locks = {}

    def _get_lock(self, symbol: str) -> asyncio.Lock:
        """Creates or retrieves a lock for a given symbol."""
        if symbol not in self._position_locks:
            self._position_locks[symbol] = asyncio.Lock()
        return self._position_locks[symbol]

    async def panic_exit(self, symbol: str, quantity_to_sell: Decimal):
        """
        Executes the 3-stage panic protocol to liquidate a position.
        This function is designed to be atomic per symbol using an asyncio.Lock.
        """
        lock = self._get_lock(symbol)

        async with lock:
            logging.warning(f"PANIC PROTOCOL INITIATED for {symbol} to sell {quantity_to_sell}.")

            remaining_quantity = quantity_to_sell
            order_id = None

            try:
                # --- STAGE 1: Snap to Mid ---
                logging.info(f"[{symbol}] STAGE 1: Placing LIMIT order at Mid-Price.")

                ticker = await self.client.get_ticker(symbol)
                mid_price = (ticker['bid'] + ticker['ask']) / 2

```

```

        order = await self.client.place_limit_order(symbol, "sell",
remaining_quantity, mid_price)
        order_id = order['id']
        logging.info(f"[{symbol}] STAGE 1 Order ID: {order_id} at price
{mid_price}.")

        await asyncio.sleep(0.5) # Wait 500ms

        status = await self.client.get_order_status(order_id, symbol)
        if status['status'] == 'filled':
            logging.warning(f"[{symbol}] PANIC RESOLVED in Stage 1.
Position closed.")
            return

        remaining_quantity -= status['filled_quantity']
        if remaining_quantity <= 0:
            logging.warning(f"[{symbol}] PANIC RESOLVED in Stage 1
(partial fill). Position closed.")
            # Ensure final cancellation for safety
            await self.client.cancel_order(order_id, symbol)
            return

        # --- STAGE 2: Limit Sweep ---
        logging.warning(f"[{symbol}] STAGE 2: Cancelling {order_id},
placing aggressive LIMIT order.")
        await self.client.cancel_order(order_id, symbol)

        ticker = await self.client.get_ticker(symbol)
# Get fresh market data
        aggressive_price = ticker['bid'] * Decimal('0.95')

        order = await self.client.place_limit_order(symbol, "sell",
remaining_quantity, aggressive_price)
        order_id = order['id']
        logging.info(f"[{symbol}] STAGE 2 Order ID: {order_id} at price
{aggressive_price}.")

        await asyncio.sleep(2.0) # Wait 2s

```

```

        status = await self.client.get_order_status(order_id, symbol)
        if status['status'] == 'filled':
            logging.warning(f"[{symbol}] PANIC RESOLVED in Stage 2.
Position closed.")
            return

        remaining_quantity -= status['filled_quantity']
        if remaining_quantity <= 0:
            logging.warning(f"[{symbol}] PANIC RESOLVED in Stage 2
(partial fill). Position closed.")
            await self.client.cancel_order(order_id, symbol)
            return

        # --- STAGE 3: Market Order ---
        logging.critical(f"[{symbol}] STAGE 3: Cancelling {order_id},
submitting MARKET order.")
        await self.client.cancel_order(order_id, symbol)

        final_order = await self.client.place_market_order(symbol,
"sell", remaining_quantity)
        logging.critical(f"[{symbol}] STAGE 3 MARKET order
{final_order['id']} submitted. PANIC sequence complete.")

    except Exception as e:
        logging.error(f"[{symbol}] CRITICAL FAILURE during panic
protocol: {e}", exc_info=True)
        # ==> ALERTING MECHANISM SHOULD BE TRIGGERED HERE (e.g.,
PagerDuty)

```

Thread Safety and Concurrency Control

The prompt asks about "thread safety." In an `asyncio` context, which is single-threaded, the analogous concern is managing concurrent tasks and preventing race conditions.

The Problem: If the `panic_exit` function is called twice for the same symbol, or if another part of the trading logic tries to modify the position while the panic is in progress, the system could end up with conflicting orders (e.g., two market sells, or a sell and a buy), leading to an incorrect final position and financial loss.

The Solution: An `asyncio.Lock` is used to ensure atomicity for operations on a specific symbol.

1. **Per-Symbol Lock:** A dictionary `self._position_locks` stores a unique `asyncio.Lock` for each symbol.
 2. **Context Manager:** The entire `panic_exit` logic is wrapped in an `async` with `self._get_lock(symbol):` block.
 3. **Mutual Exclusion:** This guarantees that only one coroutine can execute the panic logic for a given symbol at a time. If a second call to `panic_exit` for the same symbol occurs, it will wait until the first one has completely finished (or failed) and released the lock.
 4. **System-Wide Safety:** To be fully effective, *any* other part of the system that modifies a position (e.g., placing new trades, adjusting stop-losses) must also acquire the same lock before acting.
-

3. Critical Analysis

Potential Failure Modes & Edge Cases

1. **API/Network Failure:** A call to `cancel_order` could fail, leaving a "dangling" limit order on the books. If the protocol proceeds to the next stage and places another order, the system will have two active sell orders, potentially selling double the intended quantity.
 - **Mitigation:** Implement a robust `cancel_order` function with a retry mechanism. If cancellation repeatedly fails, the protocol must halt and trigger a high-priority human alert. Do not proceed to the next stage until cancellation is confirmed.
2. **Partial Fills:** The implementation correctly checks for partial fills between stages. However, a fill can occur *during* the `asyncio.sleep()` and the

subsequent `cancel_order` call. There is a race condition where the order is filled moments before the cancellation request arrives.

- **Mitigation:** After a `cancel_order` call, always make a final `get_order_status` call to get the canonical final state of that order (filled quantity, status) before proceeding.

3. **Market Halted or in Post-Only Mode:** If the exchange halts the market for the symbol, all order placement attempts will fail. The protocol's exception handling must be able to distinguish between transient network errors and terminal exchange rejections.

- **Mitigation:** The exception handling block must parse API error codes. For non-recoverable errors like "Market Halted," the protocol should stop and alert, rather than retrying.

4. **Extreme Slippage on Market Order:** In a liquidity crisis (the very reason a panic protocol might be triggered), a large market order can cause devastating slippage, executing at a far worse price than the `Bid * 0.95` limit.

- **Mitigation:** This is an accepted risk of Stage 3. The primary goal is exit, not price. However, for very large positions, the protocol could be enhanced to split the market order into smaller "iceberg" chunks, though this complicates the logic and delays the exit.

Optimizations and Enhancements

1. **Use Time-In-Force (TIF) Orders:** Many exchanges support `Immediate-Or-Cancel` (IOC) or `Fill-Or-Kill` (FOK) order types. These are superior to the `sleep-then-cancel` pattern.

- **Revised Stage 1:** Place a `LIMIT` order at Mid-Price with `time_in_force='IOC'`. The order will either fill instantly against available liquidity or be automatically cancelled by the exchange. This is faster, more atomic, and reduces network round-trips.
- **Revised Stage 2:** Similarly, the aggressive limit order could be an `IOC` order.

- **Benefit:** This simplifies the code, makes it more robust, and reduces latency, as the logic is executed on the exchange's matching engine rather than in the client application.

2. **Dynamic Pricing and Timing:** The fixed values (500ms, 2s, 5%) are rigid. A more sophisticated system could adjust them based on real-time market conditions.

- **Dynamic Price:** Instead of `Bid * 0.95`, calculate the price required to fill the order based on the current order book depth. For example, "find the price at which `quantity_to_sell` can be absorbed."
- **Dynamic Timing:** Shorten the wait times during periods of high volatility.

3. **State Machine Implementation:** For even greater robustness, the protocol can be modeled as a formal state machine (e.g., `ATTEMPTING_MID`, `ATTEMPTING_SWEEP`, `ATTEMPTING_MARKET`). This makes the logic clearer, easier to test, and more resilient to failures, as the state can be persisted and resumed.